

AI Human Activity Recognition for on-board BAS experiments

SIH26174 | Technical idea submission

A procedure-aware computer-vision assistant for reliable, explainable experiment execution in the Bharatiya Antariksh Station.

Mission context

Gaganyaan and BAS-01 will require astronauts to execute structured scientific procedures in a constrained environment. BAS-HAR observes the operator, evaluates evidence against a pre-defined YAML procedure, gives timely alerts, and produces an auditable event record for ground review.

Prepared for

Smart India Hackathon problem SIH26174 - AI HAR for On-board BAS Experiments

Authoritative project specification: SIH26174_AI_HAR_BAS_TechnicalDoc_v1.0_2026-08-27.docx

Implementation repository: C:/Users/Parth/Desktop/HAR_for_BAS

Current build

64 automated tests passing | Ruff clean | replay and live GUI paths wired

1. Mission and problem

Astronaut procedures are sequences of actions whose safety and scientific value depend on order, completion, and correct handling of objects. A generic activity classifier can recognize motion but cannot answer the operational question: did the astronaut perform the next required step, with the right object, in the right place, and for long enough to count as evidence?

BAS-HAR addresses this gap with a schema-first, procedure-aware system. Experiment knowledge stays in a validated YAML plan; perception produces generic detections and interactions; the procedure engine evaluates evidence and controls state transitions; the GUI separates astronaut-facing alerts from ground-facing detail.

Proposed value

- Explainable decisions: every transition and alert can be traced to named evidence rules.
- Resilient operation: confidence smoothing, persistence thresholds, pause handling, and explicit skip detection reduce single-frame errors.
- Experiment portability: a new procedure is a new plan, not a rewrite of the engine.
- Ground readiness: timestamped JSONL records support review, replay, and integrity verification.

Operational flow

Camera	Perception	Evidence	FSM	Alert and log
Webcam, MP4, or RTSP	YOLO, pose, hands, HOI	Persistence and geometry	Valid next steps	Voice, GUI, CRC JSONL

Design choices

The primary temporal model is a rule/procedure FSM. A learned temporal head is an explicit future version, not a hidden dependency. The edge target is the development laptop with an RTX 5050 8GB. Jetson bring-up and 3D human mesh recovery are intentionally parked until the 2D system has representative data and measured performance.

2. Architecture and implementation

BAS-HAR is organized into replaceable layers with a stable schema boundary. The same procedure engine runs in replay, live GUI, and command-line modes.

Layer	Responsibility	Current implementation
0. I/O	Acquire frames and persist evidence	OpenCV webcam/file/RTSP source, rotating MP4 segments, CRC JSONL
1. Perception	Convert frames into generic observations	YOLOv11-n wrapper, MediaPipe pose and hands, HOI heuristic
2. Temporal	Stabilize observations over time	EMA smoothing, persistence counters, pause watchdog, rate limiter
3. Procedure	Evaluate the experiment graph	Schema-driven FSM, valid successor selection, skip detection
4. Voice	Deliver concise astronaut alerts	Queued pyttsx3 worker with silence and shutdown handling
5. GUI	Show astronaut and ground views	PySide6 tabs, live QThread capture, replay mode, event tail

Schema boundary

The plan schema validates experiment identifiers, objects, regions, evidence rules, transitions, and timing constraints before execution. Logical object IDs are separate from detector class names. Region geometries are defined in camera pixel coordinates, and location rules fail validation when their geometry is absent or unknown.

Evidence semantics

- A step completes only when all of its evidence rules persist for their configured minimum frame counts.
- `in_region` requires all four axis-aligned BBox corners to be inside the named polygon.
- `outside_of` requires the object BBox not to be fully contained by the named region.
- Hand-object interactions use the detector's logical object mapping and are smoothed through the procedure engine.

Alert path

Alerts pass through confidence and persistence filters, rate limiting, and a silence window. The GUI can send the same alert to the ground list and the offline voice queue without coupling the engine to a specific user interface.

3. Demonstration procedure

The locked demonstration is the red/blue box experiment. It is intentionally small enough to explain to a judge while exercising ordered transitions, object identity, visibility, hand-object interaction, and region geometry.

#	Procedure step	Evidence signature
1	Open the big box lid	opening interaction with big_box for 10 frames
2	Take the red small box out	grasping red_box; visible outside big_box
3	Put the red small box on the table	placing red_box; visible outside big_box; fully inside table region
4	Take the blue small box out	grasping blue_box; visible outside big_box
5	Put the blue small box on the table	placing blue_box; visible outside big_box; fully inside table region
6	Close the big box lid	closing interaction with big_box for 10 frames

Judge-facing behavior

If the actor jumps to a later step, the FSM emits SKIP_DETECTED. If progress stalls, the pause watchdog marks the procedure paused. The ground view retains the evidence summary and event tail for review.

Run modes

- Replay: deterministic text DSL drives mocked perception through the same engine for golden-path and skip demonstrations.
- Live: a QThread reads the camera and hands the latest frame to the GUI timer, keeping capture off the Qt event loop.
- Engine CLI: a real source is passed through perception and the FSM, with optional CRC-enveloped event output.

Current test evidence

Area	Evidence	Result
Schema and CLI	Valid and invalid plans, references, geometry, and validator exit codes	Pass
Procedure engine	Golden path, skip, pause, silence, rate limiting, atomic sink	Pass
Region evidence	Inside, partial overlap, and outside containment cases	Pass
Capture and I/O	Source selection, rotating MP4, close behavior, CRC tamper detection	Pass
Quality gate	64 pytest tests and Ruff check	Pass

4. Validation, limitations, and safety posture

The current build proves the system contract and the end-to-end software path. It does not claim detector accuracy that has not yet been measured on representative red/blue takes.

Known limitation

The cached YOLOv11-n model is COCO-pretrained. The demo objects are not dependable COCO classes, so the real experiment requires a session-level dataset with annotated `big_box`, `small_red_box`, and `small_blue_box` classes. A sample MP4 validates wiring but correctly produces no useful demo transitions with the generic model.

Validation gates after recording

Metric	Target	Status
Object detection mAP@0.5	At least 0.85 on held-out takes	Pending real data
2D pose OKS	At least 0.75	Pending representative evaluation
Hand F1	At least 0.85	Pending representative evaluation
FSM step accuracy	At least 80 percent on held-out takes	Software tests pass; field data pending
Voice alert latency	At most 500 ms	Measure on target laptop
GUI update FPS	At least 15	Measure with tuned model
Replay determinism	Same JSONL bytes for same MP4	CRC path implemented; final benchmark pending

Risk controls

- Use session-level train/validation/test splits to prevent near-duplicate frames from leaking across evaluation.
- Keep confidence, persistence, and unknown states visible to the operator instead of forcing uncertain actions into a completed state.
- Keep a pre-recorded MP4 replay as a stage backup if a webcam or stream fails.
- Use CRC32-enveloped JSONL when events must be verified after transfer to ground systems.
- Calibrate region polygons for the selected camera resolution and placement before recording.

Terminology and scope

The project uses the term orientation-diverse augmentation. It does not claim microgravity simulation. The 3D HMR, Jetson, and learned temporal-head ideas are explicitly future or parked work, keeping the current proposal testable on the laptop target.

5. Execution roadmap

The software foundation is ready for the data and deployment loop. The remaining work is sequenced so that measured evidence arrives before optimization claims.

Next step	Deliverable	Dependency
1. Record takes	Multiple actors and camera placements; session metadata and raw chunks	Camera and physical red/blue box setup
2. Annotate	Three-class detector labels plus region calibration	Recorded takes
3. Fine-tune	Best YOLO model with held-out metrics	Dataset YAML and labels
4. Measure	FPS, latency, step accuracy, false alerts	Trained model and representative takes
5. Export	ONNX model and PyInstaller distribution	Validated trained model
6. Integrate	Real RTSP sub-stream and ground-side artifact flow	Camera/network endpoint

Ready-to-run commands

Purpose	Command
Validate plan	<code>validate-plan experiments/red_blue_box/experiment_plan.yaml</code>
Replay golden path	<code>replay-motor experiments/red_blue_box/experiment_plan.yaml scripts/samples/red_blue_box_golden.txt</code>
Record a take	<code>record-dataset --demo red_blue_box --source 0</code>
Fine-tune	<code>train-yolo datasets/red_blue_box/data.yaml --model yolo11n.pt</code>
Export	<code>export-onnx runs/train/red_blue_box/weights/best.pt --output-dir models</code>
Run GUI	<code>run-gui experiments/red_blue_box/experiment_plan.yaml --source 0</code>

Submission position	BAS-HAR is a working, inspectable MVP with an explicit evidence contract. Its key contribution is the combination of generic perception and procedure-level reasoning that can be audited, replayed, and swapped across experiments.
----------------------------	--

Source documents

This summary is derived from docs/architecture.md, docs/validation_matrix.md, docs/risk_register.md, HANDOVER.md, the validated experiment plan, and the authoritative technical document named on the cover. Performance values marked pending are deliberately not presented as achieved results.